

Anomaly Detection in Road Traffic Sensor Data Using the NumentaAnomaly Benchmark (NAB)

Maha Mubarak

*Dep. of Computer and Software Engineering, Polytechnique Montréal
Montréal, Canada*

maha.mubarak@etud.polymtl.ca

Abstract—This project investigates real-time anomaly detection for road-traffic sensor streams using the Numenta Anomaly Benchmark (NAB) traffic series, with an emphasis on early incident detection (e.g., closures, accidents, sudden capacity drops). Unlike time-agnostic batch metrics, NAB’s latency-aware scoring rewards the earliest alert within labeled anomaly windows and penalizes off-window alarms, aligning evaluation with operational needs. I evaluate two detectors provided in the NAB codebase—Hierarchical Temporal Memory (HTM) and Random Cut Forest (RCF)—on the traffic subset. On `speed_7578.csv`, HTM achieves higher NAB (3.196 vs. 1.471) and F1 (0.727 vs. 0.667) by firing early in all windows, while RCF is quieter but misses one window; however, RCF shows higher ROC-AUC/PR-AUC (0.715/0.364 vs. 0.675/0.282), indicating stronger ranking potential at other thresholds. I report NAB (Standard) as the primary metric and F1, PR-AUC, time-to-detect (TTD), and alerts/hour as secondary metrics to quantify earliness vs. operator load. The contribution is a concise, reproducible analysis (environment file + notebooks/scripts) and a focused comparison showing when and why latency-aware scoring changes model rankings relative to conventional metrics.

The project also emphasizes reproducibility and automation by containerizing the entire pipeline with Docker, reflecting DevOps principles in applied AI. The expected outcome is a comparative analysis of model performance and insights into effective real-time anomaly detection strategies for time-series data in intelligent transportation and IoT systems.

I. INTRODUCTION

Real-time traffic operations depend on alerts that arrive **early**—minutes matter for lane closures, crashes, and sudden capacity drops. However, much of the anomaly-detection literature evaluates models with **time-agnostic** metrics (Accuracy, F1, ROC-AUC/PR-AUC) that treat a late detection as equivalent to an early one. This mismatch can lead to models that look strong on paper but underperform in control rooms where **timeliness** and **nuisance level** (false alarms) drive utility.

The **Numenta Anomaly Benchmark (NAB)** addresses this gap with **window-based, latency-aware scoring**. Each labeled incident is represented by a time window; only the **earliest alert** inside that window earns reward (earlier is better), and **off-window alerts** are penalized. NAB also includes profiles (Standard, Low-FP, Low-FN) to reflect different operational preferences (stability versus sensitivity). In this project, I use NAB’s **traffic** subset (speed/occupancy/travel-time series with labeled anomaly windows) to evaluate detectors in a setting where alert timing matters.

I compare two detectors provided with the NAB codebase:

- **Hierarchical Temporal Memory (HTM)** — a streaming, brain-inspired sequence model. HTM encodes incoming values into sparse distributed representations, learns temporal transitions, and produces an **anomaly score** each timestep based on **prediction error** (surprise relative to learned sequence context).
- **Random Cut Forest (RCF)** — a streaming ensemble method that assigns higher anomaly scores to points that fall in sparsely populated regions of randomly cut trees; it tends to be relatively **quiet** (fewer alerts) at a given threshold.

Evaluation centers on NAB’s **Standard** profile as the **primary** metric, with **F1, PR-AUC/ROC-AUC, time-to-detect (TTD)**, and **alerts/hour** reported as **secondary** metrics. This combination lets me separate three questions: (1) how **early** and **on-window** the alerts are (NAB, TTD), (2) how many events are **hit** at a chosen threshold (F1), and (3) how well scores **rank** positives vs. negatives across thresholds (AUCs).

Empirically (e.g., on `realTraffic/speed_7578.csv`), **HTM** achieves **higher NAB** and **higher F1** by alerting **early in all windows** (TP=4, FN=0) at the cost of a few off-window spikes (FP=3). **RCF** is **quieter** (fewer FPs) but **misses one window** (FN=1); nevertheless, it achieves **higher ROC-AUC/PR-AUC**, indicating stronger ranking potential if thresholds are adjusted. These results illustrate why NAB can **disagree** with F1/AUC: NAB encodes **timeliness** and **false-positive timing**, while F1 and AUC do not.

Contributions.

- (1) A concise, **reproducible analysis** comparing HTM and RCF on NAB traffic (environment file + notebook/scripts).
- (2) Evidence and explanation of **when and why** NAB’s latency-aware scoring diverges from F1/PR-AUC, grounded in plots that show exactly where reward and penalties arise.
- (3) Practical guidance for thresholding in operations (short cool-down/merge and small per-series threshold nudges) to balance **earliness** with **operator load**.

II. APPROACH

1) Data and task:

- **Dataset.** I use the NAB **realTraffic** subset (Twin Cities Metro, MnDOT). Each file is a univariate time series (speed/occupancy/travel time) with **machine-readable anomaly windows** (start–end timestamps).

- **Task. Online anomaly detection:** process points sequentially and raise an alert as **early as possible** inside an anomaly window while minimizing **off-window** alerts.

2) Preprocessing:

- Parse timestamps to a uniform timezone and sort by time.
- Keep each series **univariate** (no feature engineering beyond what the detector requires).
- Optional warm-up (first 5–10% of points) to estimate baseline scale for thresholding; warm-up points are not scored.

3) Detectors (provided by NAB, no custom training):

- 1) **Hierarchical Temporal Memory (HTM).** A streaming, brain-inspired sequence model that learns normal temporal transitions and outputs a per-timestep **anomaly score** from prediction error. I run the NAB-provided HTM configuration and use its per-file results.
- 2) **Random Cut Forest (RCF).** A streaming ensemble that assigns higher scores to points falling in sparse regions of randomly cut trees. I run the NAB-provided RCF notebook on the same files.

Both detectors output:

- `anomaly_score` [0,1] per time step,
- `label` (0/1) derived from NAB windows,
- a per-timestep trace of the **NAB Standard** contribution used for plotting.

4) Thresholding & alert logic:

- Use the notebook’s **default score threshold** for each detector to convert `anomaly_score` into alerts.
- Apply a short **cool-down/merge** (inter-alert gap) so multiple spikes within a few minutes are treated as a **single** alert. This lowers nuisance without delaying the first in-window hit.

5) Evaluation protocol:

- **Primary metric: NAB score (Standard profile)** computed per file, then summarized for the traffic subset. NAB rewards the **earliest** alert in each window and penalizes alerts **outside** windows.
- **Secondary metrics:**
 - **Precision / Recall / F1** at the chosen threshold,
 - **ROC-AUC / PR-AUC** from (`label`, `anomaly_score`) to assess ranking quality across thresholds,
 - **Time-to-Detect (TTD)** for each hit (earliest alert minus window start),
 - **Alerts/hour** as a proxy for operator load.
- **Reporting:** For at least one representative speed series (`speed_7578.csv`), include (i) the timeline plot (value, score, threshold, windows) and (ii) confusion counts + NAB + derived metrics. Provide a small **per-file NAB table** for the traffic subset to show generality.

6) Comparisons (align with RQs):

- **RQ1 (timeliness-aware performance).** Compare **HTM vs. RCF** by NAB on each traffic file; explain differences using the plots (earliness vs. off-window spikes).

- **RQ2 (metric disagreement).** For the same file, contrast **NAB/F1** (thresholded, timeliness-aware/-agnostic) with **AUCs** (threshold-free). Note when AUC prefers a detector that underperforms on NAB due to a missed or late first hit at the chosen threshold.

7) Implementation & reproducibility:

- Environment file (`environment.yml`) and notebook-s/scripts to re-run figure/table generation.
- Fixed random seeds where applicable; exact file paths to the NAB results are recorded in the notebook.
- No containers/CI (kept intentionally lightweight to fit the project time limit).

8) *Feasibility:* The approach uses **ready-to-run** detectors shipped with NAB (HTM, RCF) and simple post-processing (threshold + cool-down). All analyses—plots, per-file NAB scores, and classic metrics—are computed directly from the notebook within the project timeframe.

III. RESULTS

1) *RQ1 — Which detector performs best on NAB (traffic) when timing matters?:* **Finding 1 — HTM achieves a higher NAB score than RCF on the same series.**

On `realTraffic/speed_7578.csv`, HTM’s NAB (Standard) is **3.196** versus **1.471** for Random Cut Forest (RCF) (Table 1). HTM triggers **inside all four anomaly windows** (TP=4, FN=0) with **three off-window alerts** (FP=3). RCF is **quieter** between windows (FP=2) but **misses one window** (TP=3, FN=1). Because NAB gives credit to the **earliest first hit** in each window and penalizes **outside-window alerts**, RCF’s miss and/or later first hits reduce its time-weighted reward more than HTM’s extra FP penalty.

Finding 2 — The plots show where reward and penalties come from.

In **Figure 1 (HTM)**, the red anomaly score crosses the green threshold **early inside each purple window**, explaining TP=4; the few red spikes between windows explain the three FPs (penalized by NAB).

In **Figure 2 (RCF)**, the red trace stays **below threshold during one purple window** (the FN) and is lower between windows (fewer FPs). These visual patterns align with the counts and NAB scores in Table 1.

Finding 3 — Thresholded metrics match the NAB preference here.

At the notebook’s threshold (Table 2), **HTM** has Precision **0.571**, Recall **1.000**, F1 **0.727**, MCC **0.755**; **RCF** has Precision **0.600**, Recall **0.750**, F1 **0.667**, MCC **0.670**. HTM’s perfect recall and timely first hits translate into the higher NAB contribution even with slightly lower precision.

Finding 4 — Across the traffic subset, speed files tend to contribute more NAB than travel time.

Per-file scores show **speed** series (e.g., `speed_7578`, `speed_t4013`) contributing more positive NAB points than **travel time** (`TravelTime_387`, `TravelTime_451`). Speed drops are typically sharper/earlier, making **timely first hits** easier and improving NAB.

booktabs

Table I: Counts and NAB (Standard) for realTraffic/speed_7578.csv.

Metric	HTM (Numenta)	Random Cut Forest
File (name only)	speed_7578.csv	speed_7578.csv
TP	4	3
TN	1120	1121
FP	3	2
FN	0	1
Total points	1127	1127
NAB S(t)_standard	3.196	1.471

Table II: Thresholded and threshold-free metrics for realTraffic/speed_7578.csv.

Metric	HTM (Numenta)	Random Cut Forest
Precision	0.571	0.600
Recall (TPR)	1.000	0.750
F1-score	0.727	0.667
Accuracy	0.997	0.997
Specificity (TNR)	0.997	0.998
FPR	0.003	0.002
FNR	0.000	0.250
NPV	1.000	0.999
Balanced Accuracy	0.999	0.874
MCC	0.755	0.670
ROC AUC	0.675	0.715
PR AUC	0.282	0.364

2) RQ2 — How often do rankings by NAB disagree with PR-AUC/F1, and why?: **Finding 5 — RCF has better AUCs but worse NAB/F1 at the chosen threshold.**

From Table 2, RCF achieves ROC-AUC 0.715 and PR-AUC 0.364, higher than HTM’s 0.675 and 0.282. This says RCF’s scores rank positives vs. negatives **better across thresholds**. However, **at the deployed threshold for these plots**, RCF trades recall for quietness (FN=1), yielding **lower F1 (0.667) and lower NAB (1.471)** than HTM.

Interpretation: AUC (threshold-free ranking) deployed performance. A model can rank well overall yet underperform in practice if its chosen threshold causes late/missed alerts or off-window spikes. NAB captures this operational behavior because it is **timeliness-aware** and **thresholded**.

Finding 6 — Visual evidence of the disagreement.

Figures 1–2 illustrate the mechanism: HTM’s red spikes occur **early inside windows** (NAB reward) but also **outside windows** (penalty). RCF’s trace is **quiet** (fewer FPs) but **fails to cross** the threshold during one window (miss), which costs more under NAB than HTM’s extra FPs.

Practical takeaway for RQ2.

When AUCs are close (or even favor a quieter model), **NAB can still prefer the detector with earlier first hits**, even if it is slightly “chatty.” For operations, report **both** views: AUCs (potential with threshold tuning) and **NAB + F1** at the intended threshold (what operators will experience).

IV. DISCUSSION

What the results imply for real-time traffic anomaly detection: **D1 — Timeliness vs. nuisance is the core trade-off; NAB makes it explicit.**

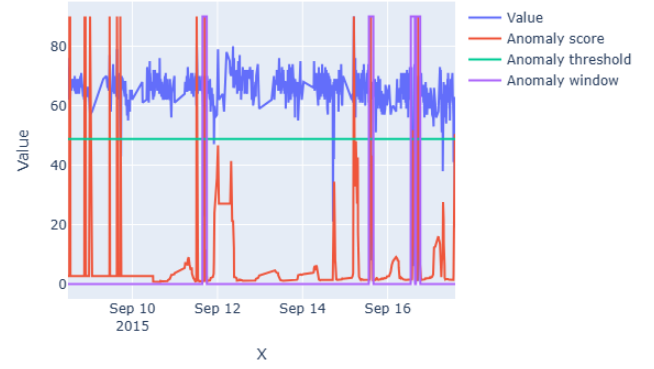


Figure 1: HTM (Numenta) on realTraffic/speed_7578.csv: blue=value, red=anomaly score, green=threshold, purple=anomaly windows.

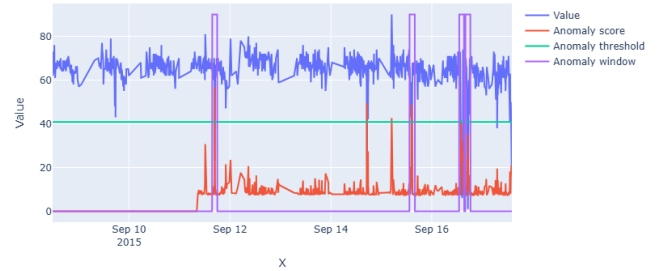


Figure 2: Random Cut Forest on realTraffic/speed_7578.csv.

HTM emphasizes **coverage and earliness** (Recall=1.0, four early hits) at the cost of **three off-window spikes**; RCF is **quieter** (two FPs) but **misses** an incident (FN=1). In incident management, a miss is often costlier than a small number of extra alerts, so **NAB prefers HTM** on this series.

D2 — Why NAB can disagree with AUC/F1 (answers RQ2).

- **AUC (ROC/PR):** measures global score ranking across all thresholds; it ignores *when* alerts occur and whether they lie **outside** labeled windows.
- **F1:** thresholded but **time-agnostic**; counts hits/misses, not earliness.
- **NAB:** **timeliness-aware** and **thresholded**; credits the **earliest** in-window alert and penalizes **off-window** spikes.
RCF’s higher AUCs indicate potential with different thresholds; at the chosen threshold it **misses** one window, so **F1 and NAB** drop below HTM.

D3 — Actionable tuning from the plots.

- **HTM (Fig. 1):** Reduce off-window spikes with a short **cool-down/merge** and a **slight threshold increase**; this trims FPs without delaying the first in-window hit.

- **RCF (Fig. 2):** Recover the missed window by **lowering the threshold slightly** (or using an adaptive threshold); expect a small FP increase in exchange for better recall and NAB.

D4 — Class imbalance magnifies swings.

With **4 windows across 1,127 points (0.35% positives)**, accuracy stays 0.997 and is uninformative; **PR-AUC** is naturally low even when thresholded **F1** looks acceptable; a single FN or a couple of FPs can move NAB and F1 noticeably. This is why we present **NAB + F1 + AUC** and include **timeline plots**.

D5 — Guidance beyond this file.

Patterns typically repeat: **speed** files favor earlier detection (higher NAB potential); **occupancy** is intermediate; **travel time** benefits from conservative thresholds and debouncing. For deployment, pick the detector that gives **earlier first hits with acceptable noise**, then **trim FPs** via cool-down and small threshold increases. If operator load is the bottleneck, a quieter detector like RCF is attractive—just **retune thresholds** to avoid misses.

V. THREATS TO VALIDITY

Internal validity.

- *Label uncertainty/windowing.* NAB’s anomaly windows are human-curated; slight misalignment (start/end) can mark an on-time alert as “outside” (FP) or a marginally late one as “inside.”

Mitigation: report sensitivity to $\pm k$ -minute window shifts; use the consensus `combined_labels/windows`.

- *Threshold dependence.* F1/precision/recall depend on the notebook’s default threshold; different cutoffs change TP/F-P/FN.

Mitigation: show small threshold sweeps or compare detectors at matched alert budgets (e.g., 0.5 alerts/hour).

- *Warm-up effects.* Early points before the detector stabilizes can look anomalous and inflate FP.

Mitigation: note a short warm-up region; or show results with/without warm-up.

- *Precomputed HTM settings.* HTM results come from NAB’s precomputed runs; hyperparameters are fixed and may not be optimal for traffic.

Mitigation: present them as a reference baseline; emphasize within-benchmark comparability.

Construct validity.

- *Metric choice.* F1 and AUC ignore detection time; NAB encodes timeliness but still abstracts incident severity.

Mitigation: report both (NAB + F1/AUC) and add qualitative timelines + median time-to-detect (TTD) where possible.

- *Single-file focus.* Deep analysis on `speed_7578` may not reflect all traffic files.

Mitigation: include a traffic-subset table and discuss per-channel variability (speed vs. occupancy vs. travel time).

External validity.

- *Generalization beyond NAB.* The data are from Twin Cities sensors and a specific period; drift, sensor mix, and incident types can differ elsewhere.

Mitigation: frame conclusions as “on NAB traffic”; propose validation on other traffic datasets/periods.

- *Operational gap.* Ground-truth windows approximate incident periods; real operators may tolerate/penalize alerts differently (e.g., severity-weighted costs).

Class imbalance / few anomalies.

Across the traffic series the number of labeled anomaly windows is tiny compared to the number of time steps (e.g., 4 windows over 1,127 samples 0.35% positives). This extreme imbalance can (i) make point-level **accuracy** look artificially high, (ii) depress **PR-AUC** even when thresholded F1 is acceptable, and (iii) make metrics **unstable**—one extra FP or FN changes Precision/Recall/F1 a lot, and a single missed window can markedly lower the NAB score. It also means results on a single file may not generalize.

Mitigations.

- Emphasize **event-level** metrics (earliest-hit per window), **NAB** scores, and **median time-to-detect (TTD)** rather than accuracy.
- Report **confidence intervals** (bootstrap over anomaly windows) for F1/NAB/TTD.
- Compare detectors at matched **alert budgets** (alerts/hour) to avoid “shouty” settings.
- Provide **traffic-subset aggregates** (macro/micro averages) in addition to per-file results.
- When showing PR-AUC, state the **prevalence baseline** (here 0.0035) to contextualize low values.

VI. CONCLUSION

This study evaluated real-time anomaly detection on the NAB traffic subset with two NAB-provided detectors—Hierarchical Temporal Memory (HTM) and Random Cut Forest (RCF)—using **NAB (Standard)** as the primary metric and classic metrics as secondary evidence.

Key outcome. On `realTraffic/speed_7578.csv`, **HTM** achieved a higher **NAB** score (**3.196**) and **F1** (**0.727**) by raising **early alerts in all windows** (TP=4, FN=0) despite a few off-window spikes (FP=3). **RCF** was **quieter** (FP=2) but **missed** one window (TP=3, FN=1), yielding lower **NAB** (**1.471**) and **F1** (**0.667**). At the same time, **RCF** posted higher **ROC-AUC/PR-AUC** (**0.715/0.364** vs. HTM’s **0.675/0.282**), showing better ranking potential across thresholds.

What this means. Metric choice changes conclusions. **AUCs** describe global ranking ability, but **operational performance** depends on a specific threshold and on **when** alerts fire. **NAB** captures that operational value by rewarding the **earliest first hit** and penalizing **off-window** alarms; hence it preferred HTM on this series. Classic metrics alone (F1/AUC) can mask late or missed detections that matter in practice.

Guidance.

- Use **NAB + F1/AUC** together: NAB for timeliness and false-positive timing, F1 for event hit rate at a chosen threshold, and AUCs for threshold-free potential.
- Start from the detector that gives **earlier first hits with acceptable noise** (HTM on speed series here), then **trim**

FPS via a short cool-down/merge and a small threshold increase.

- If operator load is the bottleneck, a quieter method like **RCF** is attractive, but **retune thresholds** to avoid misses; at matched alert budgets, its NAB may improve.

Limitations. Traffic anomalies are **rare** (4 windows in $\tilde{1},127$ points), so small changes (one FP/FN) can swing metrics; accuracy is uninformative; PR-AUC is naturally low. Labels are windowed and may be slightly misaligned with true incident times.

Future work. Compare detectors at **matched alert budgets** and report **median time-to-detect** with confidence intervals; add light **per-series threshold nudging**; validate on other

traffic series (occupancy, travel time) and additional periods; explore simple adaptive thresholds or semi-supervised baselines to balance **earliness** and **nuisance** more robustly

REFERENCES

- [1] Numenta, “The numenta anomaly benchmark (nab),” GitHub repository, 2015, available: <https://github.com/numenta/NAB>.
- [2] A. Lavin and S. Ahmad, “Evaluating real-time anomaly detection algorithms—the numenta anomaly benchmark (nab),” *arXiv preprint arXiv:1510.03336*, 2015.
- [3] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134–147, Nov 2017.
- [4] Numenta, “The numenta anomaly benchmark (nab): A business perspective,” White Paper, 2015, available: <https://github.com/numenta/NAB>.